



Application Development

Applied course notes and studio guide

Build one useful mobile-first experience

A practical LEA Labs module for turning a user need into a testable, explainable application.

Programme

Software Engineering

Module

Application Development

Level

Applied / project-led

Duration

3 weeks · 6 guided lessons

Prepared for LEA Labs learners, tutors, and programme facilitators · 2026 edition

Inside this course pack

1	Course orientation	1
1.1	Learning outcomes	1
1.2	The six-lesson route	1
1.3	Working vocabulary	1
2	Lesson 1: How apps are built	2
2.1	The central idea	2
2.2	A useful mental model	2
2.3	Architecture boundaries	2
2.4	Choosing a first release	3
2.5	Lesson 1 check	3
3	Lesson 2: Designing your app's screens	3
3.1	Start with a user journey	3
3.2	Screen anatomy	3
3.3	Platform-aware design	4
3.4	Accessibility is product quality	4
3.5	Lesson 2 check	4
4	Lesson 3: Buttons, inputs, and events	5
4.1	Interaction is a contract	5
4.2	Events flow in, state flows out	5
4.3	Form design pattern	5
4.4	Component thinking	5
4.5	Lesson 3 check	6
5	Lesson 4: Making it talk to the web	6
5.1	A network is not a function call	6
5.2	API contract literacy	6
5.3	Secure boundaries	6
5.4	Local-first thinking	7
5.5	Lesson 4 check	7
6	Lesson 5: Testing on your device	7
6.1	Testing is evidence, not theatre	7
6.2	A device test matrix	7
6.3	Usability review protocol	8
6.4	Lesson 5 check	8
7	Lesson 6: Publishing your app	8
7.1	Shipping is a learning loop	8
7.2	Release checklist	8
7.3	Demonstrating the product	9
7.4	Lesson 6 check	9
8	Capstone studio: One useful task	9
8.1	Brief	9

8.2	Required evidence	9
8.3	Assessment rubric	10
9	Quick-reference checklists	10
9.1	Before coding	10
9.2	Before testing	10
9.3	Before release	11
10	Glossary	11
11	References	11

How to use these notes

Read each lesson before studio time, then complete the build task with a real user in mind. The notes are deliberately framework-aware but not framework-dependent: the same product thinking applies whether you use React Native with Expo, native Android, native iOS, Flutter, or another responsible toolchain.

1 Course orientation

The LEA Labs promise

Application development is not primarily about memorising a framework. It is about understanding a user's context, making a small set of decisions visible, building a reliable flow, and learning from evidence. By the end of this module you will have a working mobile-first application concept, a tested core journey, and a release plan that another person can understand.

1.1 Learning outcomes

By the end of the module, a learner should be able to:

- map one meaningful user task into a focused application flow;
- translate a flow into screens, components, events, and state;
- build a mobile-first interface with clear hierarchy and feedback;
- connect one screen to remote data while handling loading, empty, and error states;
- test across device sizes and scenarios, record evidence, and prioritise fixes;
- explain a credible release plan, known limitations, and the next learning loop.

1.2 The six-lesson route

#	Lesson	Core question	Evidence
1	How apps are built	What are the parts of a modern application, and why must responsibility be separated?	Product brief + architecture map
2	Designing your app's screens	How does a user move from need to outcome with the least confusion?	User flow + screen plan
3	Buttons, inputs, and events	How does interaction change state and communicate clearly?	Core interaction
4	Making it talk to the web	How can an app be useful when a network is slow, empty, or wrong?	Remote data flow
5	Testing on your device	What evidence tells us that the experience works for real people?	Test notes + fixes
6	Publishing your app	How do we release responsibly and create the next feedback loop?	Demo + release checklist

1.3 Working vocabulary

Term	Meaning
Application	A product that helps a person complete a task through an interface, logic, data, and a release channel.
Component	A reusable interface or behaviour unit with a clear responsibility and contract.
State	The information that describes what the app currently knows or what the user is doing.

Event	An input or occurrence that may cause state to change, such as a tap, submit, response, or timeout.
Repository	A boundary that coordinates access to a data source and keeps data rules in one place.
Release	A deliberate version made available to users with known changes, checks, and rollback thinking.

2 Lesson 1: How apps are built

2.1 The central idea

A modern application is a small system, not just a set of screens. It has an experience layer that people can see, a state model that describes what is happening, a set of rules that decide what is allowed, and one or more data sources that provide or store information. The first engineering decision is therefore not “Which button library should I use?” but “Where does each responsibility belong?”

Android’s official architecture guidance emphasises separation of concerns, persistent data models, a single source of truth, and unidirectional data flow. These ideas generalise beyond Android: they reduce accidental coupling and make it easier to test what the app does when the operating system pauses, recreates, or removes part of the application process [1].

2.2 A useful mental model

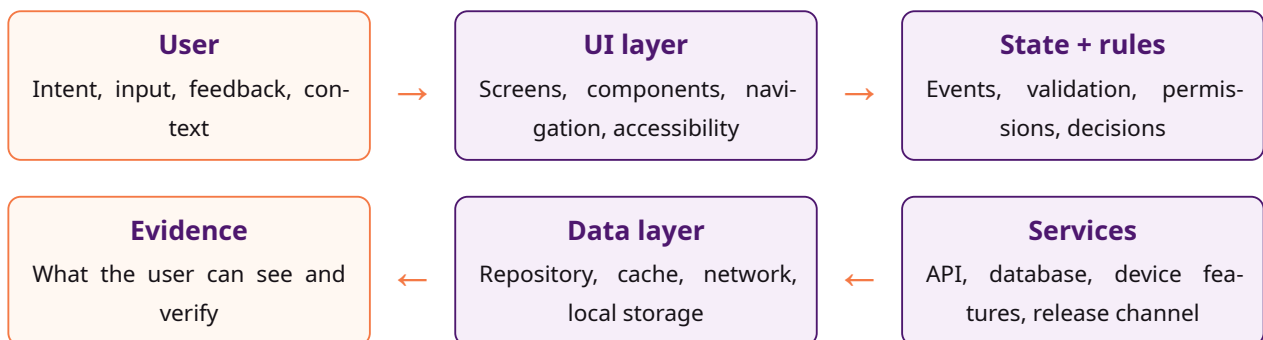


Figure 1: A responsibility map for a mobile-first application.

The diagram is intentionally simple. The user should not need to know which service or data source is behind a button. The engineering team, however, must know where decisions live. If a screen directly changes three different databases, validates input in two different ways, and mixes loading logic with layout, the product becomes difficult to reason about.

2.3 Architecture boundaries

Boundary	Owns	Does not own
UI	Rendering, layout, focus, labels, navigation events	Business rules, secrets, raw database writes
State holder	Screen state, event handling, derived view state	Pixel layout details, unrelated screens
Domain rules	Eligibility, pricing, validation, workflow decisions	Button styling, platform-specific rendering

Repository	Data access, mapping, caching, retry policy	User-facing copy and visual hierarchy
Service	Remote persistence, authentication, notifications	Assumptions about a single screen's layout

2.4 Choosing a first release

A first release should be narrow enough to finish and meaningful enough to test. Use the “one user, one job, one proof” rule:

1. **One user:** identify the person and context, not a vague market.
2. **One job:** state the task in a verb phrase, such as “save a tutoring request” or “find the next class.”
3. **One proof:** define what will show that the task worked, such as a confirmation state, a stored record, or a shareable result.

Studio task · Product brief

Write a one-page brief for an application that helps a defined user complete one meaningful task. Include the user, context, current frustration, proposed outcome, non-goals, one success measure, and three risks. Draw the responsibility map above in your own words.

2.5 Lesson 1 check

Before continuing, you should be able to explain why UI components are not a safe place for permanent application data, name the source of truth for your core record, and point to the boundary where validation and permissions are enforced.

3 Lesson 2: Designing your app's screens

3.1 Start with a user journey

A screen is not a destination; it is a moment in a journey. Begin with the user's starting context, the decision they need to make, the action they take, and the feedback they need before moving on. Avoid designing ten screens before you can describe the first successful path in five steps.

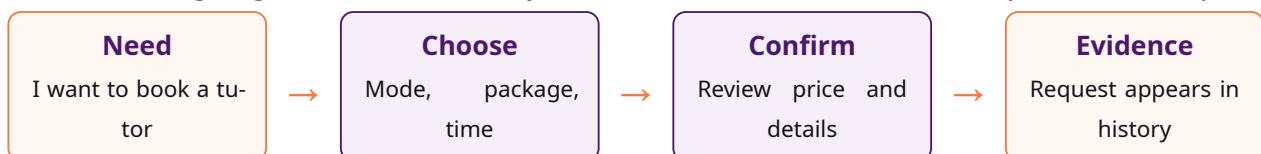


Figure 2: A focused user journey from need to outcome.

A good flow anticipates questions. What if the learner has no enrolment? What if the chosen date is unavailable? What if the request is saved but not yet confirmed? A design that shows only the happy path is incomplete.

3.2 Screen anatomy

Use hierarchy to make the next action obvious. A practical mobile screen often needs only five layers: context, title, primary content, decision controls, and feedback. The order can change, but the responsibilities should remain clear.

Layer	Question	Example for a tutor-request app
-------	----------	---------------------------------

Context	Where am I and what matters now?	Calendar title, selected course, session type
Primary content	What must I understand?	Available packages and quoted price
Decision controls	What can I do next?	Choose mode, date, time, submit request
Feedback	What happened after I acted?	Saved, awaiting review, confirmed, declined
Recovery	How do I continue when something fails?	Retry, edit, contact support, pre-serve input

3.3 Platform-aware design

Design for the platform without copying every platform convention blindly. Apple's Human Interface Guidelines provide platform-specific patterns for iOS, while Android provides its own adaptive layout and navigation guidance [7]. The transferable principle is to respect the device: allow content to reflow, support different input methods, avoid tiny targets, and make system status visible.

A mobile-first screen should be tested at narrow widths before it is stretched to a tablet. A layout that works only at one width is not responsive; it is a screenshot.

3.4 Accessibility is product quality

Accessibility means a person can perceive, operate, understand, and recover within the experience. In React Native, accessibility labels, roles, hints, states, and actions expose useful semantics to assistive technology across Android and iOS [2]. W3C guidance explains how WCAG 2.2 principles can be applied to native, mobile web, and hybrid applications [8].

Use the following design checks:

- every icon-only action has a meaningful accessible label;
- every text field has a visible or programmatically associated label;
- focus moves predictably after navigation, submission, and validation errors;
- color is not the only signal for success, warning, or failure;
- text remains readable when the user increases system font size;
- touch targets have enough space and do not depend on perfect precision;
- error messages explain what happened and how to recover.

Studio task · Flow before polish

Turn your product brief into a five-to-eight-step user flow. Identify the primary path, one empty state, one validation error, one network error, and one return path. Sketch the highest-value screen with hierarchy, accessible labels, and a visible next action.

3.5 Lesson 2 check

You should be able to justify every screen in your flow, explain what the user sees before and after the main action, and identify at least three accessibility decisions made before visual styling.

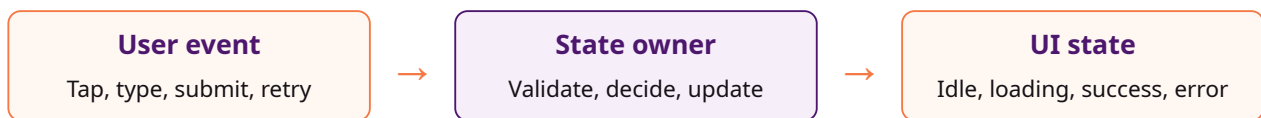
4 Lesson 3: Buttons, inputs, and events

4.1 Interaction is a contract

A control promises that something will happen. A button therefore needs an action, a visible result, and a state for moments when the action is unavailable. A text field needs a label, input rules, an error model, and a decision about when validation should run. A list item needs a clear affordance if it can be opened, edited, or deleted.

Do not treat “onPress” as the whole design. Ask what happens when the user taps twice, loses network access, submits incomplete input, rotates the device, or returns to the screen after another activity.

4.2 Events flow in, state flows out



The rendered screen is a function of current state.

Figure 3: Unidirectional flow keeps interaction explainable.

A state model becomes more reliable when impossible combinations are avoided. Instead of separate booleans such as `isLoading`, `hasError`, and `isComplete`, consider a single status value: `idle`, `loading`, `success`, or `error`. If the screen can be both “loading” and “success” at once, the state model is asking for bugs.

4.3 Form design pattern

Stage	What the app does	What the learner should verify
Initial	Shows an empty but usable field	Label, example, keyboard type, focus order
Editing	Preserves input and communicates constraints	Character limits, formatting, helpful hint
Invalid	Explains the problem near the field	Specific message, no blame, recovery path
Submitting	Disables duplicate submission and shows progress	No accidental double writes
Success	Confirms the record and next location	Saved evidence, clear next action

4.4 Component thinking

A component should be reusable because its responsibility is stable, not merely because it appears twice. Define a contract: what information it receives, what events it emits, what states it supports, and what accessibility semantics it owns. A “`TutorRequestCard`” might receive request details and emit “edit” or “cancel”; it should not secretly fetch unrelated learner records.

Studio task · Build the core interaction

Implement the highest-value interaction from your flow. Include a disabled state, a loading state, an error state, and a saved state. Keep the input, validation rule, state transition, and

confirmation copy visible in your code. Add a short decision note explaining where the source of truth lives.

4.5 Lesson 3 check

You should be able to describe the event that begins your flow, the state transition it causes, the evidence shown after success, and the recovery path when the operation fails.

5 Lesson 4: Making it talk to the web

5.1 A network is not a function call

Remote data arrives later, may be incomplete, may be duplicated, and may fail for reasons the app cannot control. A resilient application treats network communication as a state machine rather than assuming that a request immediately returns perfect data.

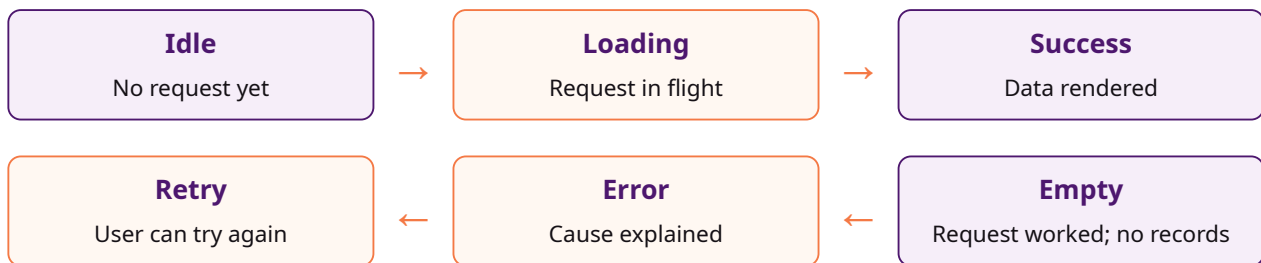


Figure 4: A practical remote-data state machine.

5.2 API contract literacy

Before writing code, read the API contract. Record the endpoint, method, authentication requirement, input shape, success response, error responses, pagination rules, and rate limits. A small contract table prevents a large amount of guesswork.

Contract item	Example	Implementation decision
Endpoint	POST /api/v1/learner/tutoring	Use a typed client function, not an untracked fetch call
Auth	Bearer session token	Never assume a user is authenticated because a screen is visible
Input	Course, mode, package, preferred time, notes	Validate required fields before sending
Success	Request record with quoted price	Show confirmation and refresh request history
Failure	401, 403, 409, 422, 500	Map each class of failure to a useful recovery message

5.3 Secure boundaries

Never place private API keys in application code. React Native's security guidance states that anything included in the delivered application can be inspected, so secrets must remain on a trusted server or be replaced with appropriately scoped public credentials [4]. Store only the minimum data the product needs. Do not log access tokens, passwords, private notes, or unredacted personal information during debugging.

Authentication and authorisation are different. Authentication answers “Who is this?” Authorisation answers “What may this user do?” A learner may be authenticated but still unable to view another learner’s tutoring request or an administrator’s official document.

5.4 Local-first thinking

An application can feel reliable even when the network is not. Preserve unsent form input, show the last known safe state when appropriate, and label stale data honestly. If offline support is part of the brief, define conflict behaviour before implementing it: what happens when the same record changes on two devices?

Studio task · One remote data flow

Connect the core screen to one remote or mock service. Implement loading, success, empty, error, retry, and authentication-required states. Write a small data contract and a security note that identifies where secrets and permission checks live.

5.5 Lesson 4 check

You should be able to show a working request-response flow, explain what the user sees for at least four non-success outcomes, and identify the server boundary that protects private data.

6 Lesson 5: Testing on your device

6.1 Testing is evidence, not theatre

Testing asks whether the product behaves as intended in a context that matters. A passing test suite is useful, but it is not proof that an unclear workflow, unreadable label, or slow screen is acceptable. Combine automated checks with scenario-based and human-centred review.

React Native’s testing guidance describes complementary layers ranging from static analysis to unit and component tests and end-to-end tests [3]. Use the lightest test that gives meaningful confidence, then add a higher-level check when different parts of the system must work together.

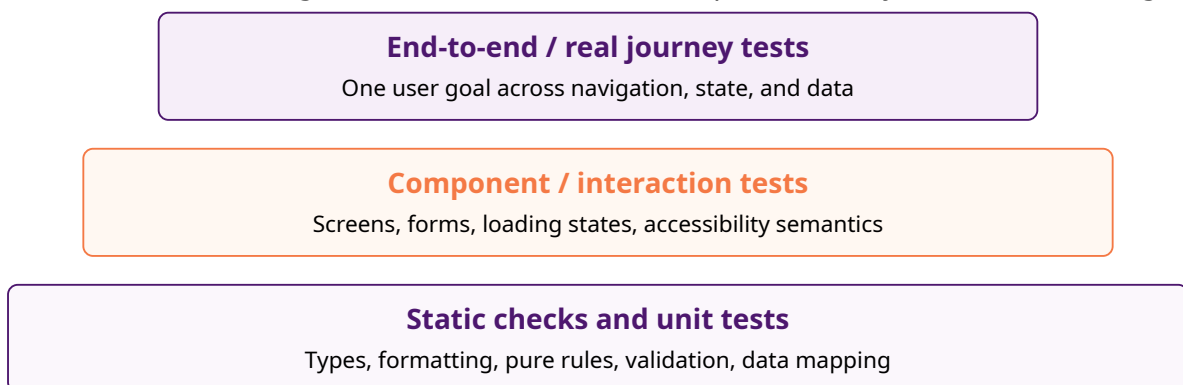


Figure 5: A practical testing stack for a small application.

6.2 A device test matrix

Context	Scenario	Expected	Evidence to record
Small phone	Core task with narrow width	No clipped text or hidden action	Screenshot + observation

Large phone / tablet	Orientation or width change	Layout reflows and state remains	Before/after notes
Slow network	Submit or load data	Progress is visible; retry is possible	Recorded timing + state
Assistive technology	Navigate and activate controls	Labels, roles, focus, feedback	Accessibility notes
Returning user	Leave and reopen screen	Important state is re-stored safely	Reproduction steps

6.3 Usability review protocol

Ask a person to complete the task without coaching. Give them the goal, not the route: “Book a face-to-face tutor session for your next course.” Observe where they hesitate, what they say aloud, and what they expect to happen. Do not defend the interface during the session. Record the context, expected result, observed result, severity, and next action.

A useful severity model is:

- **Blocker:** the task cannot be completed or data is at serious risk;
- **High:** the task can be completed only with confusion, workarounds, or repeated failure;
- **Medium:** the task works but an important part is unclear, slow, or inconsistent;
- **Low:** polish, copy, or minor visual issue that does not change the outcome.

Studio task · Five-scenario test

Test the core journey in five scenarios: ideal success, incomplete input, no data, slow or failed network, and return after leaving the screen. Fix the top three issues, rerun the affected scenarios, and attach a short iteration summary to your project.

6.4 Lesson 5 check

You should be able to show a test matrix, distinguish a component test from an end-to-end test, and explain how a real learner’s observation changed your interface.

7 Lesson 6: Publishing your app

7.1 Shipping is a learning loop

Publishing is not the moment the work becomes finished. It is the moment the product becomes observable in a wider context. A responsible release names what changed, what was checked, what is known to be limited, how a user can get support, and what will happen if the release must be withdrawn.

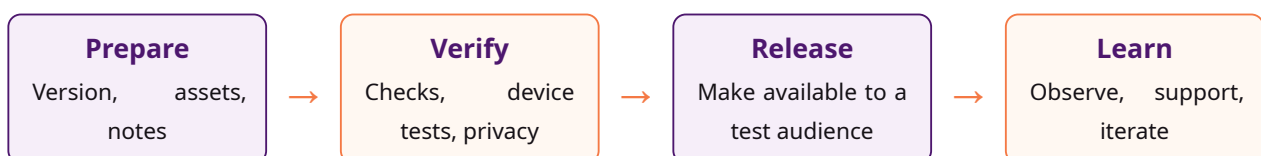


Figure 6: A lightweight release path for a learner project.

7.2 Release checklist

Area	Question	Status
------	----------	--------

Product	Can a new user understand the value and complete the primary task?	☐
Quality	Do type checks, tests, device checks, and error-state checks pass?	☐
Accessibility	Can users navigate with assistive technology and larger text?	☐
Security	Are secrets server-side, permissions explicit, and logs safe?	☐
Data	Are privacy, retention, and deletion choices documented?	☐
Support	Can a user report an issue and know what happens next?	☐
Rollback	Can the team disable or replace a problematic release?	☐
Learning	What evidence will decide the next iteration?	☐

7.3 Demonstrating the product

A professional demo is a guided argument, not a tour of every screen. In five minutes, show the user, the task, the core path, one recovery state, the evidence that data is safe or correctly persisted, and the next improvement you would make. Keep the repository, README, test notes, and release checklist available so the product can be inspected after the demo.

Studio task · Release package

Submit a working application or high-fidelity prototype, a short demo, a repository or project file, a README, a device-test matrix, an accessibility checklist, a release checklist, and a reflection describing one decision you changed because of evidence.

7.4 Lesson 6 check

You should be able to explain what is being released, who can use it, what you verified, what remains limited, and how feedback will change the next version.

8 Capstone studio: One useful task

8.1 Brief

Build an application that helps a defined user complete one meaningful task. Suitable examples include requesting a tutor session, tracking a learning goal, finding a local service, recording a household expense, organising a small event, or completing a simple youth learning activity. The feature set should remain deliberately small: one primary journey, one remote or local data flow, and enough states to feel trustworthy.

8.2 Required evidence

#	Evidence	Minimum expectation
---	----------	---------------------

1	Product brief	User, task, non-goals, success measure, risks
2	User flow and screen plan	Primary path plus empty, error, and return states
3	Working interaction	Core action, validation, loading, success, recovery
4	Data contract	Input, response, authentication, permission, failure cases
5	Test evidence	Five scenarios, observations, fixes, rerun notes
6	Release package	Demo, README, checklist, known limitations, next step

8.3 Assessment rubric

Dimension	Developing	Secure	Strong
Product clarity	User or task is vague	A focused user and outcome are clear	Scope and success measure are unusually precise
Experience	Screens feel disconnected	Primary flow is usable and states are visible	Hierarchy, accessibility, and recovery are thoughtful
Engineering	Logic is tangled or unverified	Boundaries, data flow, and tests are explainable	Architecture makes change and debugging easier
Evidence	Claims are mostly descriptive	Tests and observations support decisions	Iteration demonstrates learning from real evidence
Communication	Demo lists features	Demo explains value and limitations	Demo builds confidence and invites the next conversation

9 Quick-reference checklists

9.1 Before coding

- [] I can name the user and the task in one sentence.
- [] I know what is explicitly out of scope.
- [] I have a primary path and at least four recovery states.
- [] I know where the source of truth lives.
- [] I have identified private data and permission boundaries.

9.2 Before testing

- [] Labels, focus order, keyboard/input behaviour, and touch targets are usable.
- [] Loading, empty, error, retry, and success states are visible.

- [] The app has been checked at more than one width or device context.
- [] I can reproduce a reported issue and describe the expected result.
- [] I know which checks are automated and which require human observation.

9.3 Before release

- [] The version and change summary are recorded.
- [] Secrets and logs have been reviewed.
- [] Data retention and deletion choices are understood.
- [] A support or feedback route exists.
- [] Known limitations are written honestly.
- [] The next learning question is specific.

10 Glossary

Term	Working definition
Adaptive layout	A layout that responds to window size, orientation, or device form factor rather than assuming one fixed screen.
Affordance	A visible or semantic cue that suggests how an element can be used.
Authentication	The process of establishing who a user or service is.
Authorisation	The rules that determine what an authenticated user may access or change.
Component	A reusable unit with a clear responsibility and interface contract.
Data contract	The documented shape and behaviour of data exchanged between parts of a system.
Empty state	The interface shown when a valid request returns no records or content.
Event	An input or occurrence that may trigger a state transition.
Repository	A boundary that coordinates data access and persistence rules.
Single source of truth	One authoritative owner for a particular data type or decision.
State	The information needed to describe the app's current condition.
Unidirectional data flow	A pattern in which state moves toward the UI and user events move toward the state owner.

11 References

The following sources informed the architectural, accessibility, testing, security, and platform-aware guidance in this pack.

1. Android Developers. "Guide to app architecture." developer.android.com/topic/architecture.
2. React Native. "Accessibility." reactnative.dev/docs/accessibility.
3. React Native. "Testing Overview." reactnative.dev/docs/testing-overview.
4. React Native. "Security." reactnative.dev/docs/security.
5. React Native. "Get Started." reactnative.dev/docs/environment-setup.
6. Expo. "Create your first app." docs.expo.dev/tutorial/create-your-first-app.
7. Apple Developer. "Human Interface Guidelines." developer.apple.com/design/human-interface-guidelines.

8. W3C. "Guidance on Applying WCAG 2.2 to Mobile Applications." [w3.org/TR/wcag2mobile-22](https://www.w3.org/TR/wcag2mobile-22).

LEA LABS

Learn by building. Grow with guidance. Leave with work you can show.